

Hidden Channels in the Cloud: Two-Layer Detection of Steganography in OCI Images, from Static Bytes to Runtime Extraction

Yahia Charif
Independent Researcher
charifyahia4@gmail.com

ABSTRACT

Open Container Initiative (OCI) images are the dominant unit of software distribution in the cloud, yet their layered, content-addressed structure offers rich hiding places for steganographic payloads. We show that a payload can be smuggled through an image in ways that either (i) survive in a lower layer while being “deleted” from the runtime filesystem via an overlay whiteout, or (ii) hide inside a benign asset (e.g., an image file) that persists in the container root and is decoded at runtime. We argue that neither purely static image scanning nor purely runtime monitoring is sufficient, and present a *two-layer* detector. The static layer walks image layers and flags the structural *add-then-hide* fingerprint of whiteout smuggling; the dynamic layer uses eBPF to observe the *extraction step*—keyed on behavior, not filenames—and separates steganographic extraction from generic fileless execution using a read-to-execute byte ratio. On a controlled clean/plain/stego dataset the two layers are complementary: static detection dominates for structural carriers, dynamic detection dominates for content-embedded carriers, and their union closes the gap. We release all artifacts. These are preliminary, single-carrier findings intended as an early, reproducible baseline. This work is strictly detection-oriented and defensive.

KEYWORDS

steganography, container security, OCI images, eBPF, steganalysis, supply chain, fileless execution

1 INTRODUCTION

Containers have become the default packaging and shipping format for cloud software. An OCI image is not a monolith but a stack of *layers*, each a compressed archive of filesystem changes, combined at runtime by a union filesystem such as `overlayfs`. This design is excellent for caching and deduplication—and, we observe, convenient for hiding data.

Two properties make images attractive steganographic carriers. First, a layer can mark a file from a lower layer as deleted using a *whiteout* entry; the file vanishes from the merged view but its bytes remain physically present in the lower layer’s archive. A scanner that inspects only the final root filesystem sees nothing. Second, images routinely ship large binary assets (icons, images, fonts, sample data) whose spare capacity—trailing bytes, padding, metadata—can carry a payload without disturbing normal use.

A defender therefore faces a dilemma. Inspecting the image *at rest* (static analysis) can reveal structural anomalies but is blind to payloads that are statistically and structurally indistinguishable from legitimate assets. Observing the container *at runtime* (dynamic

analysis) can catch the act of using a hidden payload but only when it actually executes. We contend that robust detection requires *both*, and that the two techniques cover each other’s blind spots.

Contributions.

- A controlled *clean/plain/stego* image dataset and a benign, reproducible payload, isolating *delivery* as the only variable (Section 4).
- A layer-aware static detector that catches whiteout smuggling by its structural *add-then-hide* fingerprint and executables planted in data directories, rather than by signature (Section 5).
- A dynamic, eBPF-based detector for the runtime *extraction* step that is filename-agnostic and distinguishes steganographic extraction from generic fileless execution via a read-to-execute byte ratio (Section 6).
- A control ladder and preliminary evaluation showing static and dynamic coverage are complementary, producing a coverage crossover (Section 7).

Ethics and dual use. This paper is detection-first. Our payload is a harmless marker that touches a file in `/tmp`; no malware is used. We describe embedding only to the depth needed to justify and evaluate the detectors, not as an attack recipe, and we report on locally built images only. The goal is to help defenders reason about and catch a class of evasion that current image scanners largely ignore.

2 BACKGROUND

2.1 OCI images, layers, and whiteouts

An OCI image is described by a manifest that lists a configuration blob and an ordered set of layer blobs. Each layer is a tar archive (usually gzip-compressed) of *changes* relative to the layers below it: files added or modified, and files deleted. Deletion is expressed with a *whiteout*: an entry named `.wh.<name>` (or the opaque marker `.wh..wh..opq` for a whole directory). When the union filesystem merges layers, a whiteout hides the named path from all lower layers. Crucially, the lower layer is not rewritten—the “deleted” file’s bytes remain in that blob. Figure 1 illustrates the resulting asymmetry between the merged view and the physical blobs.

2.2 Steganographic carriers in images

We consider two carrier families that exploit different properties:

Structural carrier (whiteout). The payload is a real file added low and whited-out high. It is invisible in the merged filesystem but leaves a *structural* trace across layers: an add followed by a delete of the same path.

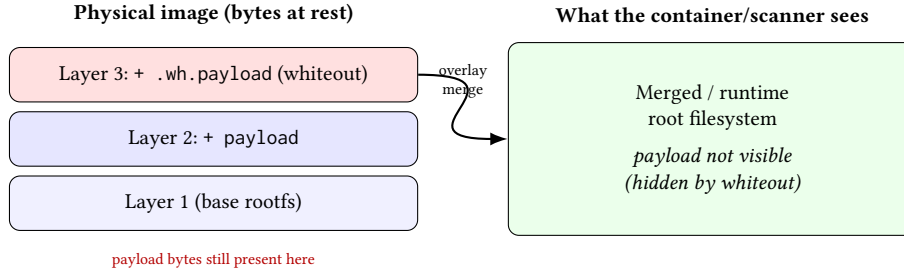


Figure 1: Whiteout smuggling. The payload is added in layer 2 and “deleted” in layer 3. It disappears from the merged runtime filesystem but persists in the layer-2 blob, invisible to roots-only scanners.

Content-embedded carrier (appended data). The payload is hidden inside a benign asset that *remains present* in the root filesystem—here, appended after the IEND chunk of a valid PNG. The image still renders; there is no whiteout and no misplaced executable. At rest it is essentially indistinguishable from a legitimate asset.

These carriers are complementary from the defender’s viewpoint, which motivates a two-layer approach.

2.3 eBPF

eBPF lets small, verified programs run in the Linux kernel and attach to events such as system-call entry/exit. Because it observes at the kernel boundary, eBPF is unaffected by user-space or union-filesystem abstractions: it sees the actual open, read, memfd_create, and execveat operations a process performs, regardless of how the container presents its filesystem. This vantage point is what lets us detect the *use* of a hidden payload even when its presence is concealed.

3 THREAT MODEL

We assume an adversary who can influence the contents of a distributed image (e.g., a compromised build step, a malicious base image, or an insider) and whose goal is to ship a payload past image-scanning defenses and use it at runtime. The defender can (a) statically scan images before deployment and (b) run a runtime sensor on hosts. We do *not* assume the adversary has escaped the container or obtained host privileges; the content-embedded carrier is decoded by a normal in-container process from an asset the container legitimately contains. Denial of service, covert network channels, and hardware side channels are out of scope.

4 DESIGN OF THE DATASET AND EXPERIMENT

To attribute any detection difference to the hiding technique alone, we hold the payload bytes constant and vary only delivery.

Payload. A tiny, freestanding (no libc) x86-64 ELF of about one kilobyte whose entire effect is to create the marker file `/tmp/stego_marker` and print one line. The same bytes are used in every variant. A freestanding ELF is used rather than a shell script because a shebang script cannot be executed filelessly from an anonymous memory file (Section 6).

Table 1: The clean/plain/stego image triplet. Payload bytes are identical; only delivery differs.

Variant	Delivery	Expected difficulty
Clean	none (control)	—
Plain	payload as an ordinary file	easy (visible)
Stego	payload hidden (whiteout / appended-data)	hard (concealed)

Table 2: Runtime control ladder. Same payload bytes throughout; each step adds one behavior. B1 isolates generic fileless execution from stego extraction.

	Mechanism	Carrier read	Decode	Fileless exec
B0	normal execve	—	—	—
B1	memfd+execveat	payload file	—	✓
C	extract + run	carrier asset	✓	✓

Image triplet. Table 1 lists the three variants built on the same base web-server image.

Control ladder for the runtime experiment. Because “run a payload from memory” is itself a generic evasion technique, we add an intermediate variant so that the runtime detector’s power can be attributed specifically to *extraction* and not merely to fileless execution (Table 2).

5 STATIC DETECTION LAYER

The static detector consumes an image (as a docker-archive tarball or an OCI layout) and iterates each layer archive *in application order*, maintaining a map of every file introduced together with metadata (executable bit, size, byte entropy). It emits findings at three severities. The key departures from a naive scanner are that it (i) never matches on payload filenames and (ii) treats whiteouts as evidence rather than noise.

Add-then-hide correlation. When the detector encounters a whiteout for path p , it checks whether p was previously *added as an executable* in a lower layer. If so, this is the structural fingerprint of whiteout smuggling—an executable that is present in the physical image yet absent from the merged filesystem—and is reported as high severity. Opaque whiteouts and ordinary deletions (no matching executable add) are downgraded.

Misplaced executables and entropy. Independently, an executable introduced into a data-only directory (e.g., `/var/log`, `/tmp`) is flagged regardless of its name, capturing the plain-delivery case without a signature; a high-entropy executable is flagged as possibly packed. These heuristics are corpus-tuned and would require an allowlist for production base images.

Baseline. We retain a deliberately naive scanner—signature-only, whiteout-blind, docker-archive-only—as the comparison baseline. It represents the rootfs/signature scanners common today and is expected to miss both concealed and renamed payloads.

6 DYNAMIC DETECTION LAYER

The dynamic layer targets the *content-embedded* carrier, which the static layer cannot see at rest. The attacker’s runtime behavior forms a distinctive chain:

```
openat(asset) → read → [decode] →  
memfd_create → write → execveat.
```

The process opens a benign asset, reads it, decodes the hidden region, writes the result into an anonymous in-memory file (`memfd`), and executes it directly with `execveat` and `AT_EMPTY_PATH`—i.e., *filelessly*, leaving no on-disk artifact.

Separating extraction from fileless execution. Fileless execution alone is a known, generic technique that runtime tools already surface. To identify *steganographic* behavior specifically, we observe that an extractor *reads much more than it executes*: it consumes a whole carrier asset but runs only the small decoded region. We therefore compute, per process, the ratio of bytes read to bytes written into the executed `memfd`. A ratio near 1:1 indicates plain fileless execution (variant B1); a large ratio indicates extraction from a carrier (variant C). The detector (an eBPF/bpf trace program) is entirely filename-agnostic: it keys on the `memfd-to-execveat` relationship and on this byte ratio, never on payload names.

Provenance caveat. The byte ratio is a per-process *sum*, a proxy for provenance rather than true data-flow taint. It establishes “a fileless execution preceded by disproportionately large reads,” not “these exact bytes came from that file.” Full taint tracking is future work (Section 9).

7 EVALUATION

We report results for the two carriers. Static results are verdicts of the two scanners over the image triplet and, for generalization, over six base images spanning databases and language runtimes; dynamic results are byte-ratio measurements captured live from the eBPF detector as each control-ladder variant executes.

7.1 Static verdicts

Table 3 contrasts the naive baseline with the layer-aware detector. The baseline misses the concealed payload and, because the plain payload is not named like its signature, misses that too—illustrating signature brittleness. The layer-aware detector flags the plain case (misplaced executable) and the whiteout case (add-then-hide), while leaving the clean image untouched.

Table 3: Static detection over the image triplet. “miss” for the baseline on Plain reflects a payload name outside its signature set.

Image	Naive baseline	Layer-aware (ours)
Clean	clean	clean
Plain	miss	detect (misplaced exec)
Stego (whiteout)	miss	detect (add-then-hide)

Table 4: Generalization of the layer-aware static detector. Plain and Stego are malicious (want detection); Clean is a false-positive check (want none). All verdicts as desired.

Base image	Class / distro	Clean	Plain	Stego
postgres:latest	db / debian	clean	✓	✓
redis:alpine	db / alpine	clean	✓	✓
mariadb:latest	db / ubuntu	clean	✓	✓
python:3.12-slim	runtime / debian	clean	✓	✓
node:20-alpine	runtime / alpine	clean	✓	✓
golang:1.22-alpine	runtime / alpine	clean	✓	✓

Generalization across base images. To check that detection is not specific to a single base, we repeated the clean/plain/stego construction on six widely used images spanning databases and language runtimes over three distributions (Table 4). The layer-aware detector flagged all 12 malicious variants (6 plain via misplaced executable, 6 stego additionally via add-then-hide) and raised *zero* findings on the six clean bases. Detection therefore transfers across base images, and the location/entropy heuristics did not misfire on real database and runtime images.

7.2 Dynamic byte ratio

We measured the ratio live with the eBPF detector attached to the host kernel (run in a privileged container so it observes the host’s syscalls), detonating each control-ladder variant in turn. B0 (normal `execve`) produced no event, correctly ignored as it uses no anonymous memory file. B1 read 1904 B (its 1072 B payload plus loader header reads) and executed 1072 B, a 1:1 ratio flagged as plain fileless execution. C read 18497 B (the carrier asset) but executed only the 1072 B decoded payload, a 17:1 ratio—well above the alerting threshold and cleanly separated from B1 (Figure 2). The detector keys only on the `memfd-to-execveat` relation and this ratio; it never inspects payload names.

7.3 Coverage crossover

The central claim is complementarity, and it holds at both carriers we measured. For the *structural* (whiteout) carrier, the static detector fires (add-then-hide; Tables 3, 4) while nothing is extracted at container runtime, so the dynamic layer does not apply. For the *content-embedded* (appended-data) carrier, the static detector reports the image clean—the carrier is a valid asset in a normal path and the extractor is an ordinary binary, leaving no structural trace—while the dynamic detector fires on the extraction (Figure 2). Neither layer covers both carriers alone; their union does (Figure 3).

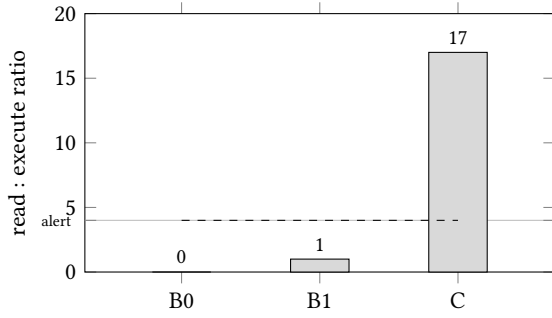


Figure 2: Read-to-execute byte ratio per control-ladder variant, measured live via eBPF. B0 does no fileless execution (shown as 0). B1 (plain fileless) sits at 1:1; C (stego extraction) at 17:1. The dashed line is the alert threshold.

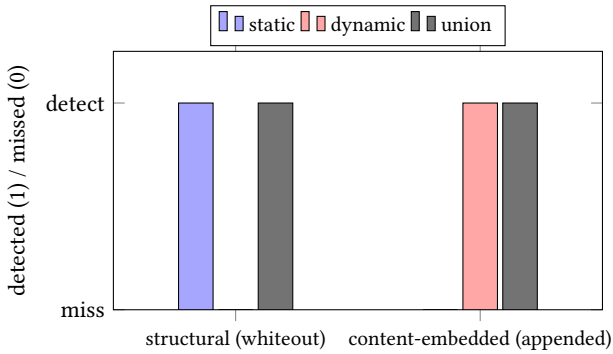


Figure 3: Measured coverage at the two carriers. The static (layer-aware) detector catches the structural carrier but misses the content-embedded one; the dynamic (eBPF) detector does the reverse; their union catches both. Each bar is a real scan or live trace, not an interpolation.

This is a measured two-point result, not an interpolated curve: each cell corresponds to an actual scan or trace.

8 RELATED WORK

Malicious-image and supply-chain scanners inspect image contents for known-bad files, secrets, and vulnerable packages, typically over the merged filesystem or per-layer file listings; they are effective against visible artifacts but, as our baseline illustrates, are weak against concealment that leaves no signature. Runtime security tools such as Falco and Tetragon detect suspicious syscall and process behavior generically—including fileless execution—but do not, to our knowledge, distinguish steganographic *extraction* from other fileless activity, which is precisely the gap our byte-ratio signal targets. Classical steganalysis focuses on statistical detection within a single medium; our contribution is to place detection at the two structural seams that containers introduce—image layers and the runtime syscall boundary—rather than within a single file. A full quantitative comparison against Falco and Tetragon on the same samples is planned for the extended version.

9 LIMITATIONS

These results are preliminary: one carrier per family and a benign payload. The static heuristics (data-directory location lists, entropy threshold) are tuned to the corpus; although they produced no false positives on the six clean base images we tested (Section 7), a production deployment over a large image population would still require allowlisting to bound the false-positive rate. The dynamic byte-ratio signal is a provenance proxy, not data-flow taint, and can in principle be evaded by laundering the carrier read and the execution across separate processes; per-process correlation would then need to become cross-process lineage tracking. We demonstrate the eBPF detector firing live on the control ladder, but a large-scale evaluation of its false-positive rate on representative production workloads (e.g., runtimes that legitimately use anonymous memory files) remains future work. Finally, we study appended-data and whiteout carriers; padding, gzip-metadata, and file-ordering carriers remain to be evaluated.

10 CONCLUSION

Container images give adversaries two very different places to hide a payload: in the physical bytes of a lower layer that the runtime pretends to have deleted, and inside a benign asset that is decoded only when it runs. We showed that a single vantage point cannot cover both, and that a static layer-aware scanner and a dynamic eBPF sensor are complementary—the former catching structural add-then-hide smuggling, the latter catching runtime extraction by its read-to-execute byte ratio, both without relying on payload names. The union of the two closes the coverage gap on our controlled dataset. We release the dataset, detectors, and control ladder to serve as an early, reproducible baseline for detecting hidden channels in OCI images.

Availability. All code and the image-building scripts are available in the accompanying repository.

REFERENCES

- [1] The Open Container Initiative. *OCI Image Format Specification*. <https://github.com/opencontainers/image-spec>.
- [2] The Linux Kernel. *Overlay Filesystem Documentation*. <https://www.kernel.org/doc/html/latest/filesystems/overlayfs.html>.
- [3] B. Gregg. *BPF Performance Tools*. Addison-Wesley, 2019.
- [4] The Falco Project (CNCF). <https://falco.org>.
- [5] Cilium/Isovalent. *Tetragon*. <https://github.com/cilium/tetragon>.
- [6] N. Provos and P. Honeyman. *Hide and Seek: An Introduction to Steganography*. *IEEE Security & Privacy*, 2003.